Bitácora Lab S11 · Persistencia políglota y caché

Claudia Alin Rojas Jiménez

1 · El punto de partida (Parte 0)

Antes de tocar código, cada consulta del saldo era una lectura a Postgres: diez consultas dieron diez valores distintos de `leidaEn` y Redis no se usaba. Primero comprobé que Redis y MongoDB contestaran, creé las cuentas y transferí 200 de Ana a Beto.

```
docker compose exec -T redis redis-cli ping
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments()'
curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000001","titular":"Ana","saldoInicial":1000}'   # y Beto 500,
Carla 300
curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json'
\
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":200}'
```

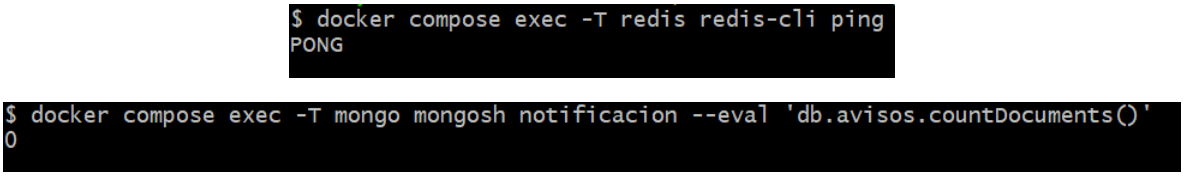


Figura 1. Redis contesta PONG y la colección avisos está vacía.

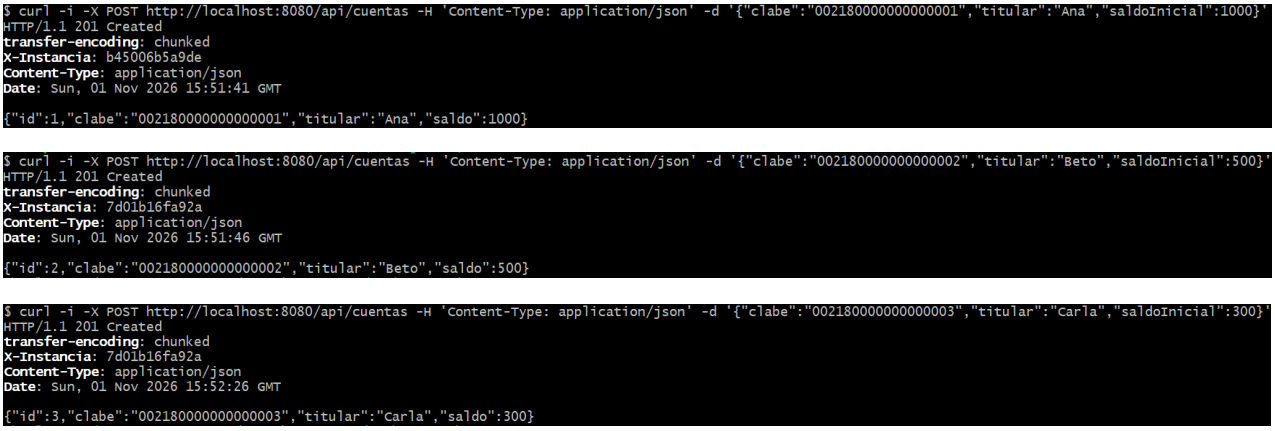


Figura 2. Las tres altas responden 201 Created.

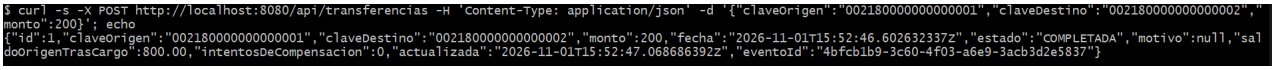


Figura 3. La transferencia de 200 de Ana a Beto queda COMPLETADA; Ana queda en 800.

Luego pedí el saldo de Ana diez veces y conté las lecturas a Postgres en el log.

```
for i in $(seq 1 10); do curl -s http://localhost:8080/api/cuentas/002180000000000001; echo;
done
docker compose logs cuenta | grep -c "leida de Postgres"
docker compose exec -T redis redis-cli INFO stats | grep keyspace
```

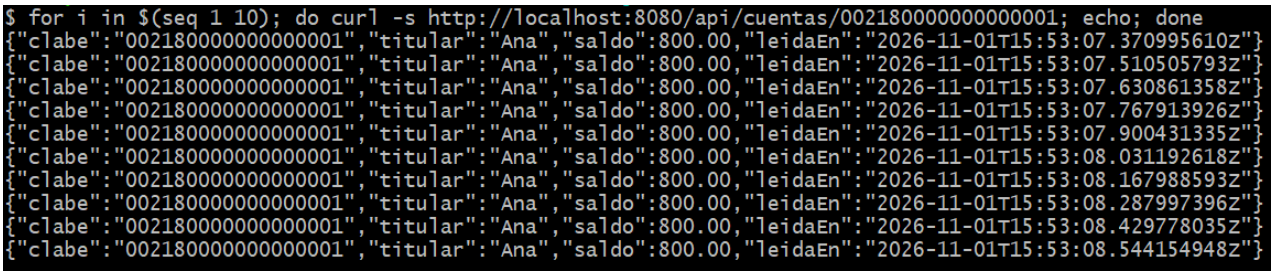


Figura 4. Diez consultas, diez valores distintos de leidaEn: cada una leyó Postgres.

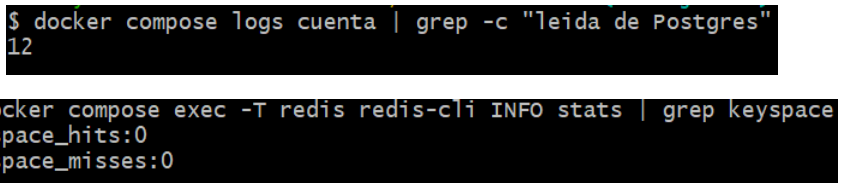


Figura 5. El log cuenta 12 lecturas (las diez consultas más dos de la transferencia) y Redis tiene 0 hits y 0 misses.

Conclusión: cada vez que el cliente abre la pantalla del saldo, Postgres trabaja, aunque el saldo no haya cambiado.

2 · La caché del saldo en Redis (Parte 1)

Con cache-aside, seis consultas atendidas por las tres réplicas de cuenta leyeron Postgres una sola vez, y una transferencia borró la copia después del commit.

El código clave

Pieza	Qué hace
`@EnableCaching` en `ConfiguracionCache`	Sin él, Spring ignora `@Cacheable` y `@CacheEvict` sin avisar.
`entryTtl(30 s)`	Cada copia caduca sola. Sin TTL, un saldo cambiado por fuera de la app se quedaría viejo para siempre.
`Jackson2JsonRedisSerializer<CuentaVista>`	Guarda la copia como JSON legible, no con la serialización binaria de Java. Usa el `ObjectMapper` de Spring, que ya sabe escribir el `Instant` de `leidaEn`.
`transactionAware()`	El `DEL` de `@CacheEvict` espera al commit de la transacción.
`@Cacheable(key = "#clabe")` en `ConsultaDeSaldo.consultar`	La lectura para mostrar. En un hit contesta Redis y el método no corre; en un miss lee Postgres y Spring guarda `cuentas::<clabe>`.
`@CacheEvict(key = "#clabe")` en `CuentaService.cargar` y `abonar`	Cuando el saldo cambia, la copia se borra, no se actualiza.

Lo que no toqué: `CuentaService.consultar` es la lectura que autoriza el cargo y siempre va a Postgres. Por eso la consulta cacheable vive en otra clase: una muestra, la otra decide. Además, un `@Cacheable` llamado desde su misma clase no pasa por el proxy de Spring y se ignora.

```
docker compose up --build -d cuenta
for i in 1 2 3 4 5 6; do curl -s -D - http://localhost:8080/api/cuentas/002180000000000001 |
grep -iE 'x-instancia|leidaEn'; done
docker compose logs --since 30s cuenta | grep -c "leida de Postgres"
docker compose exec -T redis redis-cli GET cuentas::002180000000000001
docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
```

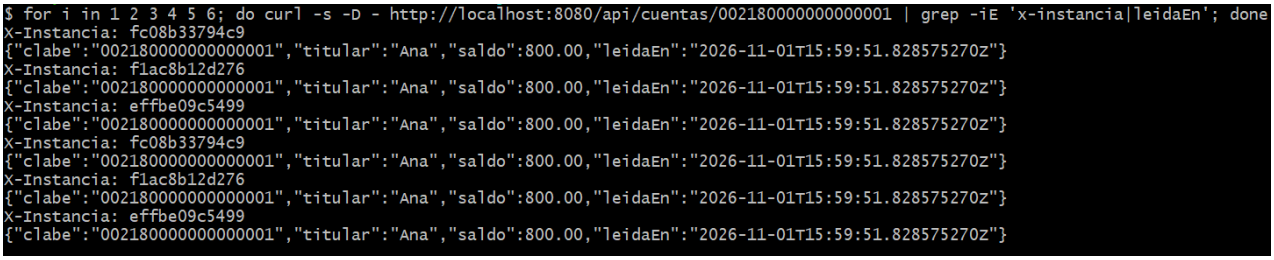


Figura 6. X-Instancia rota entre las tres réplicas, pero leidaEn es el mismo en las seis respuestas.

```
$ docker compose logs --since 30s cuenta | grep -c "leida de Postgres"
1
```

Figura 7. Una sola lectura a Postgres en los últimos 30 s: lo que guardó una réplica lo leyeron las otras dos.

En una primera corrida el conteo dio 0 porque corrí el `grep --since 30s` más de 30 s después del único miss; el comportamiento es el mismo:

```
$ for i in 1 2 3 4 5 6; do curl -s -D - http://localhost:8080/api/cuentas/002180000000000001 | grep -iE 'x-instancia|leidaEn'; done
X-Instancia: f1ac8b12d276
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-01T15:59:12.274187261Z"}
X-Instancia: fc08b33794c9
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-01T15:59:12.274187261Z"}
X-Instancia: effbe09c5499
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-01T15:59:12.274187261Z"}
X-Instancia: f1ac8b12d276
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-01T15:59:12.274187261Z"}
X-Instancia: fc08b33794c9
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-01T15:59:12.274187261Z"}
X-Instancia: effbe09c5499
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-01T15:59:12.274187261Z"}
```

Figura 8. Primera corrida: seis respuestas con un mismo leidaEn.

```
$ docker compose exec -T redis redis-cli GET cuentas::002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-01T15:59:51.828575270Z"}
```

```
$ docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
8
```

Figura 9. En Redis está el JSON de CuentaVista y le quedan 8 s de vida (TTL de 30 o menos).

La copia se borra al transferir

Con `redis-cli MONITOR` en otra terminal consulté a Ana, transferí 10 a Beto, revisé `EXISTS` y volví a consultar.

```
docker compose exec -T redis redis-cli MONITOR      # terminal 2
curl -s http://localhost:8080/api/cuentas/002180000000000001
curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json'
\
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10}'
docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001
curl -s http://localhost:8080/api/cuentas/002180000000000001
```

```
$ docker compose exec -T redis redis-cli MONITOR
OK
1793549106.478189 [0 127.0.0.1:45776] "ping"
1793549109.673048 [0 172.18.0.14:40206] "GET" "cuentas::002180000000000001"
1793549109.676914 [0 172.18.0.14:40206] "SET" "cuentas::002180000000000001" "{\"clabe\":\"002180000000000001\",\"titular\":\"Ana\",\"saldo\":790.00,\"leidaEn\":\"2026-11-01T16:05:09.676426720Z\"}" "PX" "30000"
1793549110.055009 [0 172.18.0.14:40206] "GET" "cuentas::002180000000000001"
1793549110.060436 [0 172.18.0.11:34266] "GET" "cuentas::002180000000000002"
1793549110.064892 [0 172.18.0.11:34266] "SET" "cuentas::002180000000000002" "{\"clabe\":\"002180000000000002\",\"titular\":\"Beto\",\"saldo\":710.00,\"leidaEn\":\"2026-11-01T16:05:10.064314171Z\"}" "PX" "30000"
1793549110.082509 [0 172.18.0.12:42642] "DEL" "cuentas::002180000000000001"
1793549110.238570 [0 172.18.0.14:40206] "DEL" "cuentas::002180000000000002"
1793549110.967830 [0 127.0.0.1:45780] "EXISTS" "cuentas::002180000000000001"
1793549111.355603 [0 172.18.0.11:34266] "GET" "cuentas::002180000000000001"
1793549111.359292 [0 172.18.0.11:34266] "SET" "cuentas::002180000000000001" "{\"clabe\":\"002180000000000001\",\"titular\":\"Ana\",\"saldo\":780.00,\"leidaEn\":\"2026-11-01T16:05:11.358753371Z\"}" "PX" "30000"
1793549111.512226 [0 127.0.0.1:45794] "ping"
```

Figura 10. MONITOR durante la transferencia: GET/SET del miss, comprobaciones de la saga, los dos DEL después del commit, EXISTS y el nuevo SET con 780.00.

El orden en Redis es el esperado:

1. `GET` + `SET … PX 30000`: mi consulta, un miss que llena la caché con 790.00.
2. `GET` de Ana y `GET`/`SET` de Beto: las comprobaciones de la saga.
3. `DEL cuentas::…001` y `DEL cuentas::…002`: después del commit del cargo y del abono.
4. `EXISTS` → 0: la copia ya no está.
5. `GET` + `SET` con **780.00**: la siguiente consulta trae el saldo nuevo.

3 · Mis medidas

La caché casi no cambió la latencia (~1.5–2 ms menos en la mediana), pero bajó las lecturas a Postgres de 100 a 0 o 1 por cada 100 consultas.

Hit ratio

```
docker compose exec -T redis redis-cli CONFIG RESETSTAT
# A: cien consultas seguidas
for i in $(seq 1 100); do curl -s -o /dev/null
http://localhost:8080/api/cuentas/002180000000000001; done
# B: lo mismo, con una transferencia de 1 peso de Ana a Beto cada diez consultas
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
  if [ $((i % 10)) -eq 0 ]; then curl -s -o /dev/null -X POST
http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' -d
'{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'; fi
done
docker compose exec -T redis redis-cli INFO stats | grep keyspace
```

Escenario	Hits	Misses	Hit ratio
A · 100 consultas seguidas	99	1	**99.0 %**
B · con una transferencia cada 10 consultas	100	20	**83.3 %**

En A solo falla la primera consulta. En B cada una de las 10 transferencias borra la copia de Ana y la de Beto, y la siguiente lectura de cada uno (la mía o la comprobación de la saga) es un miss: unos dos misses por transferencia. Los hits suben a 100 porque la saga también consulta `GET /cuentas`.

```
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK
```

```
$ for i in $(seq 1 100); do curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001; done
```

```
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:99
keyspace_misses:1
```

Figura 11. Escenario A: 99 hits y 1 miss.

```
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK
```

```
$ for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
  if [ $((i % 10)) -eq 0 ]; then
    curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180000000000001","claveDestino":"002180
000000000002","monto":1}'
  fi
done
```

```
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:100
keyspace_misses:20
```

Figura 12. Escenario B: 100 hits y 20 misses.

Latencia y lecturas a Postgres

```
lat() { for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n'
http://localhost:8080/api/cuentas/002180000000000001; done \
  | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms
p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'; }
antes=$(docker compose logs cuenta | grep -c "leida de Postgres"); lat; despues=$(docker
compose logs cuenta | grep -c "leida de Postgres"); echo "lecturas a Postgres: $((despues-
antes))"
CACHE_TIPO=none docker compose up -d cuenta   # sin caché; al final: docker compose up -d
cuenta
```

Corrida (100 consultas)	Promedio	Mediana	p95	Lecturas a Postgres
Con caché 1	8.22 ms	7.74 ms	10.15 ms	1
Con caché 2	7.93 ms	7.65 ms	10.00 ms	0
Con caché 3	7.51 ms	7.26 ms	8.61 ms	1
Con caché 4	7.72 ms	7.13 ms	9.40 ms	0
Sin caché 1	15.73 ms*	9.49 ms	13.59 ms	100
Sin caché 2	9.36 ms	9.14 ms	10.66 ms	100
Sin caché 3	9.52 ms	8.78 ms	11.45 ms	100

* La primera corrida sin caché incluye el calentamiento de la JVM recién reiniciada. La corrida "con caché 2" no la capturé.

En mi laptop Postgres tiene la tabla en memoria y está a un salto de red, igual que Redis, así que la diferencia de tiempo es chica. Lo que sí cambia es cuánto trabaja Postgres.



Figura 13. Con caché: medianas de 7.1 a 7.7 ms y 0 o 1 lecturas a Postgres.

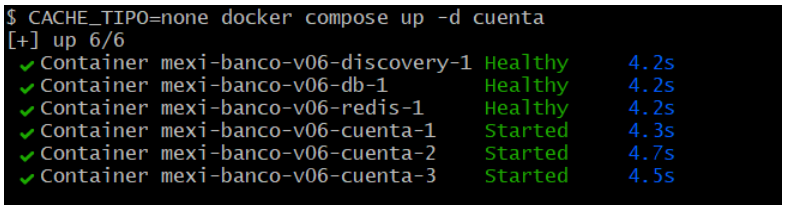


Figura 14. Se reinicia cuenta con CACHE_TIPO=none.

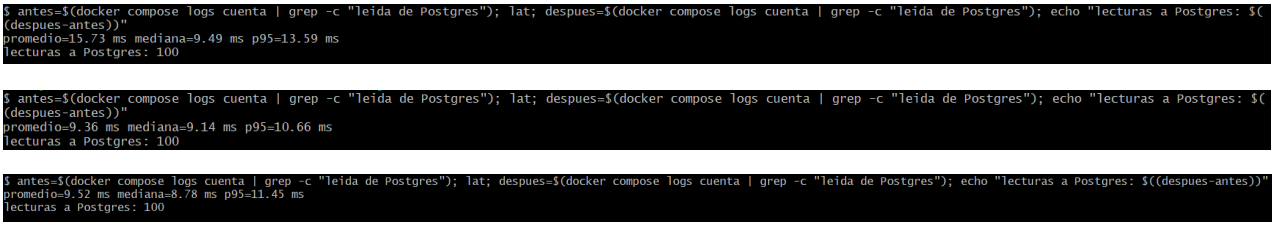


Figura 15. Sin caché: medianas de 8.8 a 9.5 ms y 100 lecturas a Postgres por corrida.

4 · Los avisos en MongoDB (Parte 2)

Los avisos se mudaron a la colección `avisos`: cada transferencia deja dos documentos con la forma de su canal, el endpoint devuelve los últimos 10 y el índice `clabe_fecha` hace que la consulta examine 10 documentos en vez de todos.

El código clave

Pieza	Qué hace
`BitacoraDeAvisos.documento()`	Arma un `org.bson.Document` con `_id = eventoId:clabe`, el dinero en `Decimal128` (nunca `double`) y la fecha del evento. Un `switch` sobre la interfaz sellada `Contacto` agrega los campos de cada canal; si faltara uno, no compila.
`registrar()`	Inserta los avisos uno por uno y cuenta los nuevos. Si Mongo responde `DuplicateKeyException`, ese aviso ya estaba: no es error y no se cuenta.
`crearIndice()`	Al arrancar crea `clabe_fecha = {clabe: 1, fecha: -1}`, con la misma forma de la consulta.
`ultimos()`	`Query` con `clabe` igual, orden por `fecha` descendente y `limit(10)`.
`ConsumidorDeTransferencias`	Una línea: guarda con `bitacora.registrar(evento)`. Los avisos se mudan, no se copian: escribir en Postgres y en Mongo sería una doble escritura sin transacción común.

El `_id` no es solo el `eventoId` porque una transferencia genera dos avisos, uno para quien envía y otro para quien recibe. Inserto uno por uno porque un insert de varios se detiene en el primer repetido, y una reentrega debe poder guardar el aviso que faltó.

```
docker compose up --build -d notificacion
docker compose logs notificacion | grep avisos
```

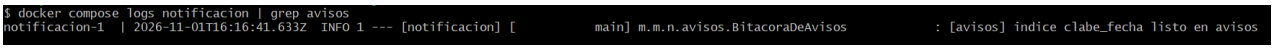


Figura 16. Al arrancar, notificación crea el índice clabe_fecha.

Las tres formas

Una transferencia a Beto y otra a Carla dejan cuatro avisos: dos SMS de Ana, un CORREO de Beto y un PUSH de Carla. Nadie hizo `ALTER TABLE`: el esquema lo aplica quien lee, como en la segunda consulta, que solo trae los documentos con `telefono`.

```
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})'
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono:
{$exists: true}}, {clabe: 1, canal: 1, telefono: 1})'
```

```
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})'
[
  {
    _id: '6016e759-41d8-46fa-80a8-501d6f396fc5:002180000000000001',
    eventoId: '6016e759-41d8-46fa-80a8-501d6f396fc5',
    transferenciaId: Long('14'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('200'),
    saldo: Decimal128('570.00'),
    fecha: ISODate('2026-11-01T16:22:29.974Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 200, saldo resultante 570.00'
  },
  {
    _id: '6016e759-41d8-46fa-80a8-501d6f396fc5:002180000000000002',
    eventoId: '6016e759-41d8-46fa-80a8-501d6f396fc5',
    transferenciaId: Long('14'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('200'),
    saldo: Decimal128('930.00'),
    fecha: ISODate('2026-11-01T16:22:29.974Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 200, saldo resultante 930.00'
  },
  {
    _id: 'bc27862a-9a70-447a-9f1c-c7fb2fc5e5c5:002180000000000001',
    eventoId: 'bc27862a-9a70-447a-9f1c-c7fb2fc5e5c5',
    transferenciaId: Long('15'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('50'),
    saldo: Decimal128('520.00'),
    fecha: ISODate('2026-11-01T16:22:41.923Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 50, saldo resultante 520.00'
  },
  {
    _id: 'bc27862a-9a70-447a-9f1c-c7fb2fc5e5c5:002180000000000003',
    eventoId: 'bc27862a-9a70-447a-9f1c-c7fb2fc5e5c5',
    transferenciaId: Long('15'),
    clabe: '002180000000000003',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('50'),
    saldo: Decimal128('350.00'),
    fecha: ISODate('2026-11-01T16:22:41.923Z'),
    canal: 'PUSH',
    token: 'fcm:demo-002180000000000003',
    titulo: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 50, saldo resultante 350.00'
```

Figura 17. Los avisos de las transferencias 14 y 15: SMS con telefono y texto, CORREO con correo, asunto y cuerpo, PUSH con token, titulo y cuerpo.

```
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}}, {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: '6016e759-41d8-46fa-80a8-501d6f396fc5:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: 'bc27862a-9a70-447a-9f1c-c7fb2fc5e5c5:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Figura 18. Solo los avisos con teléfono: los dos SMS de Ana.

Los últimos 10

Después de doce transferencias de 1 peso, el endpoint devuelve exactamente 10 avisos de Beto, todos CORREO, de la transferencia 27 a la 18.

```
for i in $(seq 1 12); do curl -s -o /dev/null -X POST
http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":1}';
done
curl -s http://localhost:8080/api/notificaciones/0021800000000000002
```



Figura 19. El endpoint por el gateway: 10 avisos, del más nuevo al más viejo.

El índice y los dos explain

```
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe:
"0021800000000000002"}).sort({fecha: -1}).limit(10).explain("executionStats")'
# lo mismo con .hint({$natural: 1}) antes de .explain(...) para no usar el índice
```

Plan	Etapas	Documentos examinados
Con índice	`LIMIT ← FETCH ← IXSCAN (clabe_fecha)`	10
Sin índice (`hint({$natural: 1})`)	`SORT ← COLLSCAN`	todos: 28 en la corrida original, 30 al capturar

Con el índice Mongo salta directo a las entradas de Beto ya ordenadas por fecha. Sin él recorre la colección completa y ordena en memoria: con millones de avisos sería leer 10 contra leer millones.

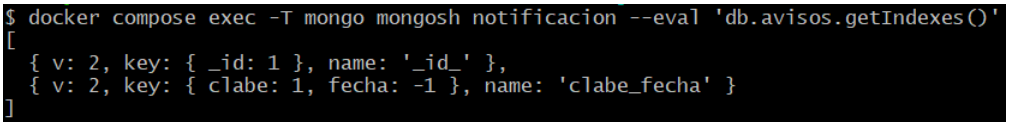


Figura 20. Índices de la colección: _id_ y clabe_fecha.

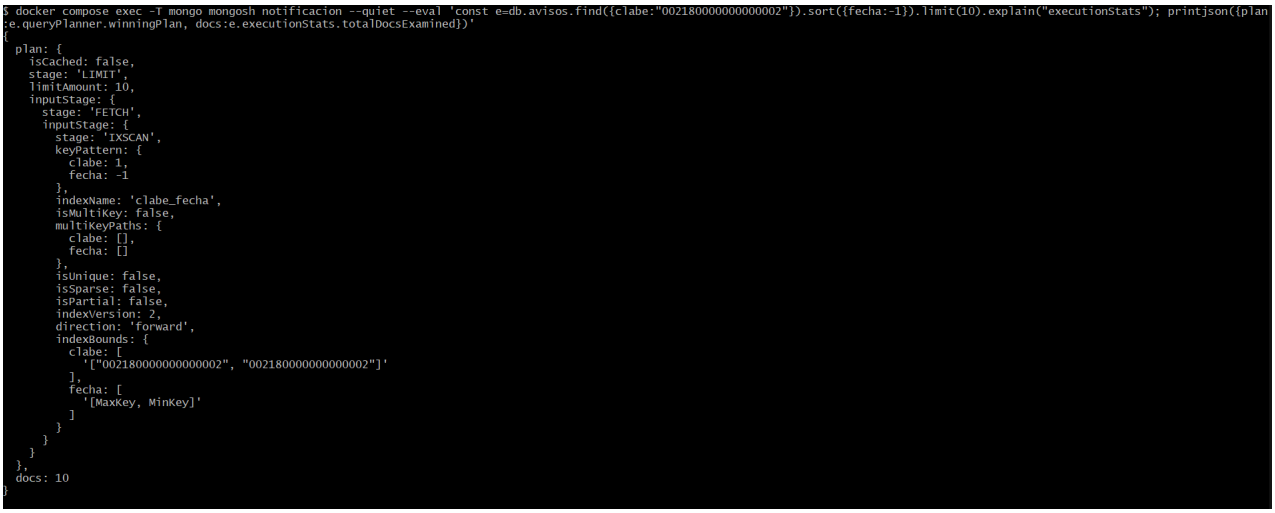


Figura 21. Plan con índice: IXSCAN y 10 documentos examinados.

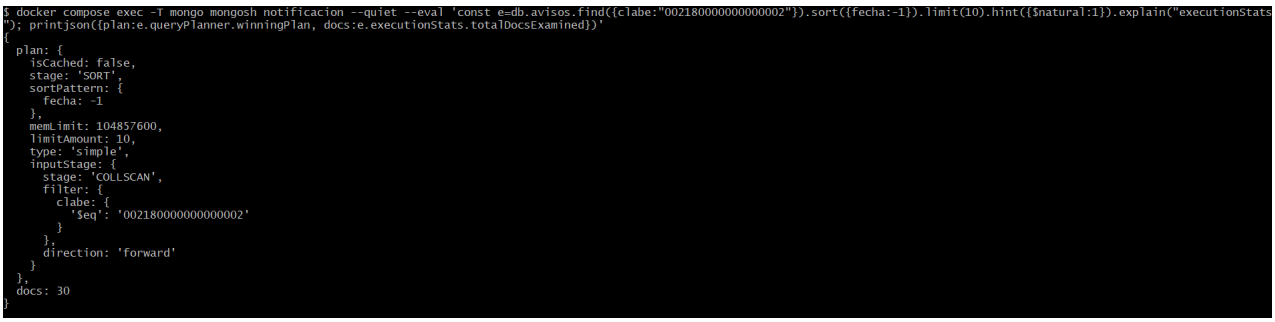


Figura 22. Plan sin índice: SORT sobre COLLSCAN. Al capturar, la colección ya tenía 30 avisos (los 2 del evento duplicado de la Parte 3).

5 · El evento duplicado (Parte 3)

El mismo evento publicado dos veces dejó 2 avisos, no 4: el `_id` hizo de llave única sin una tabla aparte. Publiqué un `TransferenciaRealizada` fijo dos veces desde la API de RabbitMQ, con CLABEs inventadas.

```
curl -u mexibanco:mexibanco -X POST
http://localhost:15672/api/exchanges/%2F/transferencias/publish \
  -H 'Content-Type: application/json' \
  -d '{"properties":
{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada
","payload_encoding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-8444-
555555555555\", ... }"}'   # dos veces
docker compose exec -T mongo mongosh notificacion --eval
'db.avisos.countDocuments({eventoId: "11111111-2222-4333-8444-555555555555"})'
docker compose logs notificacion | grep -E "ya estaba|9001"
```

Las dos publicaciones respondieron `{"routed":true}`. El primer mensaje registró 2 avisos nuevos. El segundo armó los mismos dos `_id`, Mongo rechazó ambos inserts y `registrar` los contó como *ya estaba*, con 0 avisos nuevos. Los dos llegaron con `redelivered=false`: para el broker eran mensajes distintos, y lo que evitó el duplicado fue el `_id`.



Figura 23. Dos avisos para ese evento, no cuatro.

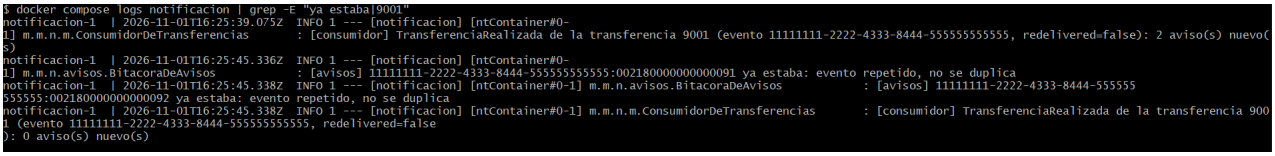


Figura 24. El segundo intento: "ya estaba: evento repetido, no se duplica" y 0 avisos nuevos.

6 · El dato viejo (Parte 4)

La caché mostró un saldo que ya no existía durante unos 30 s, pero la transferencia de 100 se rechazó con 422: la caché engañó a la pantalla, no a la decisión. Consulté a Ana (508.00 quedó en la caché) y cambié su saldo por fuera de la app, directo en Postgres, sin pasar por el `@CacheEvict`.

```
curl -s http://localhost:8080/api/cuentas/002180000000000001
docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 5 where
clabe = '002180000000000001'"
for i in $(seq 1 8); do date +%T; docker compose exec -T redis redis-cli TTL
cuentas::002180000000000001; \
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo; sleep 5; done
# en otra terminal, mientras la caché dice 508:
curl -s -w '\nHTTP %{http_code}\n' -X POST http://localhost:8080/api/transferencias -H
'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":100}'
```

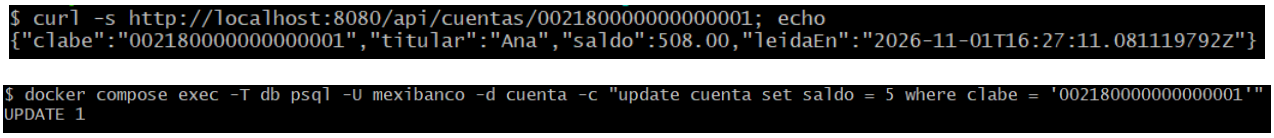


Figura 25. Ana en la caché con 508.00 y el UPDATE directo en Postgres a 5.

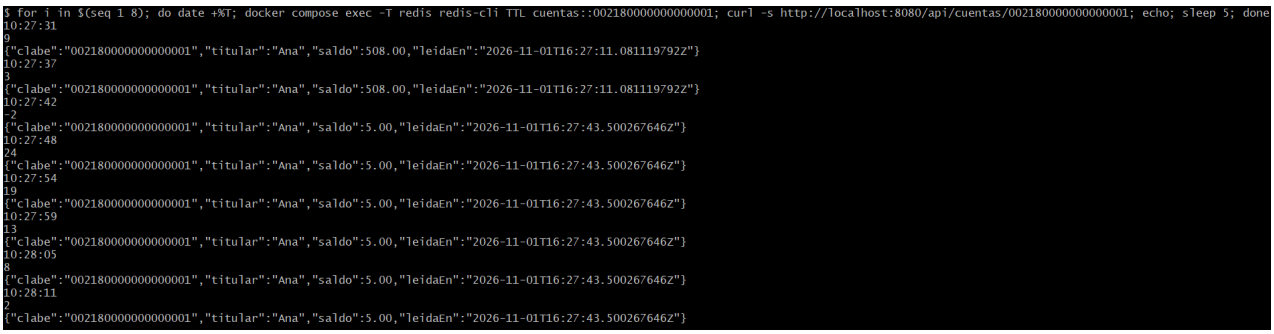


Figura 26. Cada 5 s: el TTL baja 9 → 3 → −2 y solo entonces aparece 5.00 con un leidaEn nuevo.

¿Cuántos segundos vio el cliente un saldo que ya no existía? Hasta que caducó la copia, a lo más el TTL de 30 s. `leidaEn` lo cuenta: la copia vieja se leyó a las 16:27:11 y la siguiente lectura real fue a las 16:27:43, 32 s después.



Figura 27. La transferencia de 100 mientras la pantalla decía 508: 422, saldo insuficiente.

¿Qué contestó la transferencia y por qué? 422 "Saldo insuficiente". `cargar` autoriza con `CuentaService.consultar`, que lee Postgres (5.00), no la caché (508.00). Al terminar regresé el saldo de Ana a 508.00.

7 · Preguntas de cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo?

Porque cada aviso tiene una forma distinta según el canal: SMS con `telefono` y `texto`, correo con `correo`, `asunto` y `cuerpo`, push con `token`, `titulo` y `cuerpo`. En una tabla serían columnas casi siempre vacías o tres tablas; en Mongo cada documento lleva solo lo suyo. Además se escriben una vez, nunca se actualizan y se leen enteros por CLABE, y con el índice `clabe_fecha` la consulta examinó 10 documentos. El saldo es lo contrario: cambia todo el tiempo, se protege con `@Version`, guarda la Idempotency-Key en la misma transacción y nunca debe aprobar un cargo sin fondos. Esas garantías las da Postgres.

2. ¿Qué hit ratio midieron en A y en B, y qué compraste con la caché?

A: 99 % (99 hits, 1 miss). B: 83.3 % (100 hits, 20 misses). Baja porque cada transferencia borra la copia de Ana y la de Beto, y la siguiente lectura de cada uno falla. La latencia bajó poco (mediana de ~9 ms a ~7.4 ms), pero las lecturas a Postgres pasaron de 100 a 0 o 1 por cada 100 consultas. Compré sobre todo **capacidad**: Postgres deja de atender las lecturas para mostrar y queda libre para los cargos y abonos.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?

En la Parte 4 la caché decía 508.00 mientras Postgres tenía 5.00, y la transferencia de 100 se rechazó con 422 porque el cargo lee Postgres. Si leyera la caché la habría aprobado y Ana quedaría en −95: dinero que no existe. La caché puede ir hasta 30 s atrás; por eso solo puede mostrar, nunca decidir.

4. ¿Por qué `DEL` de la copia y no `SET` con el saldo nuevo? ¿Y por qué después del commit?

- **DEL y no SET:** con `SET`, dos movimientos concurrentes en réplicas distintas pueden escribir sus saldos en desorden y dejar en Redis el más viejo, sin que nadie lo corrija hasta el TTL. Con `DEL` no se escribe ningún valor; la siguiente lectura trae la verdad de Postgres.
- **Después del commit:** si se borrara antes, una consulta podría llegar entre el `DEL` y el commit, leer el saldo todavía viejo y volver a dejarlo en la caché 30 s. Y si la transacción hiciera rollback, el borrado sobraría. El costo de ese orden lo vi en el reto: el `DEL` ocurre con el dinero ya movido, así que una falla ahí no puede tumbar la respuesta.

8 · Reto: Redis se cae

Con Redis apagado, cuenta ya responde 200 en el GET y 201 en la transferencia, los saldos cuadran y el log dice qué operación de caché falló. Llegar ahí tomó tres intentos, y en el primero se crearon 5 pesos.

Antes (8.1)

Con la versión de la Parte 1 y los saldos en 508.00 y 942.00, apagué Redis. El GET del saldo respondió 500 y la transferencia también, aunque Postgres seguía perfecto: una copia para mostrar tumbó la consulta y la saga.

```
docker compose exec -T db psql -U mexibanco -d cuenta -c "select clabe, saldo from cuenta
where clabe in ('002180000000000001','002180000000000002') order by clabe"
docker compose stop redis
curl -s -w '\nHTTP %{http_code}\n' http://localhost:8080/api/cuentas/002180000000000001
curl -s -w '\nHTTP %{http_code}\n' -X POST http://localhost:8080/api/transferencias -H
'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":5}'
docker compose logs --since 1m cuenta | grep -iE "redis|RedisConnection" | head -5
```

```
$ docker compose exec -T db psql -U mexibanco -d cuenta -c "select clabe, saldo from cuenta where clabe in ('002180000000000001','002180000000000002') order by clabe"
       clabe        | saldo
--------------------+--------
 002180000000000001 | 508.00
 002180000000000002 | 942.00
(2 rows)
```

```
$ docker compose stop redis
[+] stop 1/1
 ✔ Container mexi-banco-v06-redis-1 Stopped
```

Figura 28. Saldos reales antes del reto y Redis apagado.

```
$ curl -s -w '\nHTTP %{http_code}\n' http://localhost:8080/api/cuentas/002180000000000001
{"timestamp":"2026-11-01T16:47:58.064+00:00","status":500,"error":"Internal Server Error","path":"/cuentas/002180000000000001"}
HTTP 500
```

```
$ curl -s -w '\nHTTP %{http_code}\n' -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":5}'
{"type":"about:blank","title":"Internal Server Error","status":500,"detail":"El servicio cuenta respondio 500","instance":"/transferencias","servicio":"cuenta"}
HTTP 500
```

Figura 29. Con Redis apagado, el GET del saldo y la transferencia responden 500.

```
$ docker compose logs --since 1m cuenta | grep -iE "redis|RedisConnection" | head -5
cuenta-2  | 2026-11-01T16:47:53.036Z  INFO 1 --- [cuenta] [xecutorLoop-1-1] i.l.core.protocol.ConnectionWatchdog     :
 Reconnecting, last destination was redis/172.18.0.2:6379
cuenta-3  | 2026-11-01T16:47:53.022Z  INFO 1 --- [cuenta] [xecutorLoop-1-1] i.l.core.protocol.ConnectionWatchdog     :
 Reconnecting, last destination was redis/172.18.0.2:6379
cuenta-3  | 2026-11-01T16:47:53.032Z  WARN 1 --- [cuenta] [ioEventLoop-4-2] i.l.core.protocol.ConnectionWatchdog     :
 Cannot reconnect to [redis/<unresolved>:6379]: Connection refused (connect failed): redis/172.18.0.2:6379
cuenta-2  | 2026-11-01T16:47:53.049Z  WARN 1 --- [cuenta] [ioEventLoop-4-2] i.l.core.protocol.ConnectionWatchdog     :
 Cannot reconnect to [redis/<unresolved>:6379]: Connection refused (connect failed): redis/172.18.0.2:6379
cuenta-2  | 2026-11-01T16:47:58.930Z  INFO 1 --- [cuenta] [xecutorLoop-1-7] i.l.core.protocol.ConnectionWatchdog     :
 Reconnecting, last destination was redis/<unresolved>:6379
```

Figura 30. Log de cuenta: Lettuce no puede reconectar con Redis.

Por qué un CacheErrorHandler no alcanza

Con `transactionAware()`, `@CacheEvict` no borra en el momento: **registra** el borrado para `afterCommit`. El `CacheErrorHandler` envuelve ese registro, que no falla; el `DEL` real corre después, fuera de su alcance. Si Redis está caído, la excepción sale de `cargar()` o `abonar()` cuando el dinero **ya se movió**, cuenta responde 500 y la saga cree que la operación falló. Por eso atrapé el error en el `RedisCacheWriter`, la pieza que de verdad habla con Redis y que también ejecuta el `DEL` de después del commit.

Tres intentos

Intento	Qué agregué	Resultado
1	`EscritorDeCacheTolerante`: GET que falla = miss; SET o DEL que falla = log; tras una falla, 5 s sin intentar Redis	**Se crearon 5 pesos.** El abono a Beto hizo commit, pero su DEL tardó ~4 s en fallar; la saga cortó a los 2 s y devolvió los 5 a Ana.
2	Lettuce con `REJECT_COMMANDS` y `connect-timeout: 500ms`	Seguía tardando ~3 s: resolver el nombre `redis` con el contenedor detenido tarda eso y ningún timeout del cliente lo cubre. 503, sin mover dinero.
3	Límite propio: cada operación en un hilo aparte (máximo 4, sin cola), a lo más **300 ms**	GET 200 en 0.62 s, transferencia 201 en 1.48 s, saldos cuadran.

El intento 1 quedó como salida de terminal, sin captura:

```
{"status":502,"detail":"No se pudo abonar a la CLABE 002180000000000002, asi que se
devolvieron 5 ...",
 "transferencia":31,"estado":"COMPENSADA","motivo":"... /abonos\": Read timed out"}

 transferencia-31-abono      | 002180000000000002 | ABONO | 5.00 | 947.00   <- sí se aplicó
 transferencia-31-cargo      | 002180000000000001 | CARGO | 5.00 | 503.00
 transferencia-31-devolucion | 002180000000000001 | ABONO | 5.00 | 508.00   <- la saga
devolvió de más

cuenta-3 16:50:29.216  POST /cuentas/0021800000000000002/abonos atendida por acbdc8aa9b62
cuenta-3 16:50:33.136  [cache] fallo DEL ... en Redis (RedisConnectionFailureException)
```

Corregí el saldo de Beto a 942.00 con un UPDATE; el asiento del abono 31 sigue en el servicio movimiento como consecuencia del error.

El corazón del intento 3:

```
private <T> T intentar(String operacion, String cache, byte[] clave, Supplier<T> accion, T
siFalla) {
    if (enPausa()) return siFalla;                                  // tras una falla, 5 s
sin intentar Redis
    Future<T> pendiente = null;
    try {
        pendiente = HILOS.submit((Callable<T>) accion::get);
        T resultado = pendiente.get(LIMITE_MS, TimeUnit.MILLISECONDS);   // 300 ms máximo
        if (sinRedisHasta.getAndSet(0) != 0) log.info("[cache] Redis contesta otra vez");
        return resultado;
    } catch (TimeoutException | RejectedExecutionException e) {
        if (pendiente != null) pendiente.cancel(true);
        fallo(operacion, cache, clave, e);                          // log + pausa
        return siFalla;                                             // GET = miss; SET/DEL
= se omiten
    } catch (ExecutionException e) { ... } catch (InterruptedException e) { ... }
}
```

El riesgo que acepté: un `DEL` que no se pudo hacer deja una copia vieja a lo más por el TTL (30 s), el mismo límite de la Parte 4. Como Redis guarda solo en memoria, al reiniciarse vuelve vacío.

Después, con Redis apagado (8.3)

```
docker compose up --build -d --no-deps cuenta
time curl -s -w '\nHTTP %{http_code}\n' http://localhost:8080/api/cuentas/002180000000000001
time curl -s -w '\nHTTP %{http_code}\n' -X POST http://localhost:8080/api/transferencias -H
'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":5}'
docker compose logs --since 1m cuenta | grep "\[cache\]"
```

Entre pruebas, el circuit breaker de `transferencia` (S08) rechazó un intento con "Circuito abierto" por los timeouts anteriores; se cerró solo a los 10 s.

```
$ docker compose exec -T db psql -U mexibanco -d cuenta -c "select clabe, saldo from cuenta where clabe in ('002180000
000000001','002180000000000002') order by clabe"
       clabe        | saldo
--------------------+--------
 002180000000000001 | 508.00
 002180000000000002 | 942.00
(2 rows)
```

Figura 31. Saldos antes de la prueba: 508.00 y 942.00.

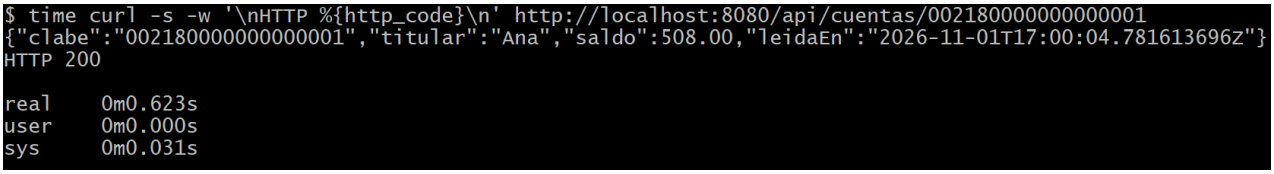


Figura 32. GET con Redis apagado: 200 en 0.62 s.

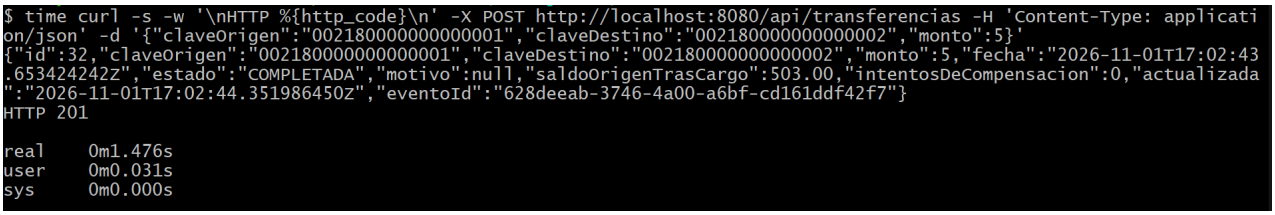


Figura 33. Transferencia con Redis apagado: 201 COMPLETADA en 1.48 s.

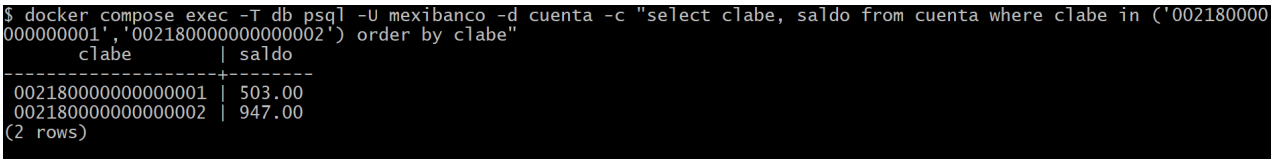


Figura 34. Saldos después: 503.00 y 947.00. Cuadran.

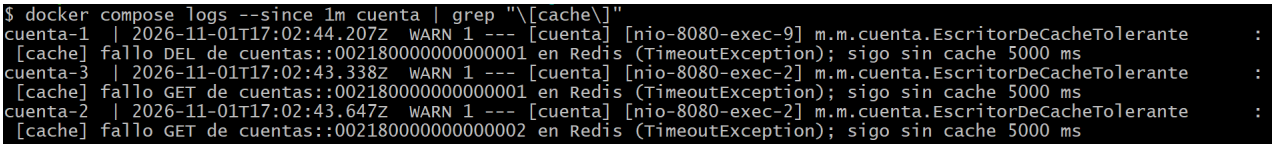
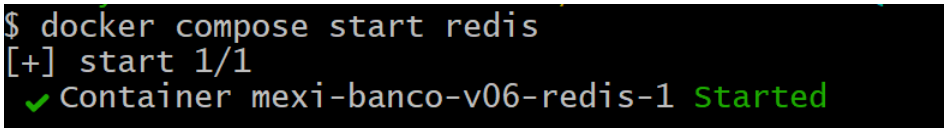


Figura 35. El log dice qué operación falló: GET y DEL, por TimeoutException.

Redis vuelve (8.4)

Al encender Redis, las tres réplicas registran *Redis contesta otra vez*, tres consultas devuelven el mismo `leidaEn` y la copia tiene TTL. No hizo falta reiniciar cuenta.

```
docker compose start redis
sleep 6
for i in 1 2 3; do curl -s http://localhost:8080/api/cuentas/002180000000000001; echo; done
docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
docker compose logs --since 1m cuenta | grep "\[cache\]"
```



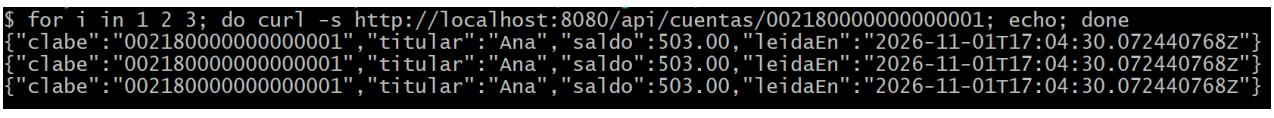


Figura 36. Redis encendido otra vez; tres consultas con el mismo leidaEn.

```
$ docker compose exec -T redis redis-cli TTL cuentas::00218000000000000124
```

```
$ docker compose logs --since 1m cuenta | grep "\[cache\]"
cuenta-1  | 2026-11-01T17:04:30.260Z  INFO 1 --- [cuenta] [nio-8080-exec-4] m.m.cuenta.EscritorDeCacheTolerante      : [cache] Redis contesta otra vez
cuenta-3  | 2026-11-01T17:04:30.069Z  INFO 1 --- [cuenta] [nio-8080-exec-6] m.m.cuenta.EscritorDeCacheTolerante      : [cache] Redis contesta otra vez
cuenta-2  | 2026-11-01T17:04:30.441Z  INFO 1 --- [cuenta] [nio-8080-exec-5] m.m.cuenta.EscritorDeCacheTolerante      : [cache] Redis contesta otra vez
```

Figura 37. TTL de 24 s y las tres réplicas notan que Redis volvió.

- "La caché solo muestra" no basta: también tiene que poder fallar sin estorbar, y en tiempo acotado.
- Atrapar la excepción no alcanza si llega tarde: un timeout que excede el de quien llama convierte una operación confirmada en un "fallo", y la compensación crea dinero.
- Los timeouts del cliente no cubren todo (DNS); un límite propio en la frontera es más confiable.